BREVET DE TECHNICIEN SUPÉRIEUR

SERVICES INFORMATIQUES AUX ORGANISATIONS

Rapport de stage

Consolidation et modernisation de la lib helpers PHP interne

Effectué par BHOWMIK Shukdeb

Du 15 décembre 2025 au 06 Février 2026

Tuteur de stage : M. Julien NAPOLI

Superviseur académique : M. ARGENTIERE

Établissement : Lycée Louis Armand – BTS SIO (2ème année)

Entreprise d'accueil : Les Jardins Numériques (Paris)

REMERCIEMENTS

Je tiens tout d'abord à remercier l'entreprise **Les Jardins Numériques** et son président, **M. Joseph HAN**, pour m'avoir accueilli au sein de leur structure et pour la confiance qu'ils m'ont témoignée dès le début de ce stage.

Je remercie tout particulièrement mon maître de stage, **M. Julien NAPOLI**, pour la qualité exceptionnelle de son encadrement. Au-delà des compétences techniques en PHP, Git et administration serveur que j'ai pu acquérir grâce à lui, j'ai énormément apprécié sa pédagogie, sa patience et sa bienveillance tout au long de mon apprentissage. Ses conseils sur les bonnes pratiques professionnelles ("Clean Code", Rigueur, Sécurité) me seront précieux pour la suite de ma carrière.

J'exprime également ma gratitude envers mon professeur référent, **M. ARGENTIERE**, ainsi qu'à toute l'équipe pédagogique du **Lycée Louis Armand**, pour leur suivi et pour la solide formation théorique qui m'a permis d'aborder ce stage avec sérénité.

Enfin, je remercie ma famille et mes proches pour leur soutien inconditionnel.

SOMMAIRE

BREVET DE TECHNICIEN SUPÉRIEUR 1
SERVICES INFORMATIQUES AUX ORGANISATIONS 1
REMERCIEMENTS 2
SOMMAIRE 3
INTRODUCTION 4
I. PRÉSENTATION DE L'ASSOCIATION 5
1.1 Les Jardins Numériques : Une mission sociale et éducative 5
1.2 L'équipe et l'organisation technique 5
II. MISE EN PLACE DE L'ENVIRONNEMENT TECHNIQUE 6
2.1 Un nouveau contexte : Du local au distant 6
2.2 Le défi de la connexion SSH 6
2.3 L'environnement de développement : VS Code et Remote SSH 7
2.4 La réalité du Terminal : Droits et Sudo 7
III. DEVELOPPEMENT ET MODERNISATION 8
3.1 L'analyse du problème : L'héritage technique ("Legacy Code") 8
3.2 Ma mission : "Diviser pour mieux régner" 8
3.3 La réalisation technique : Typage et Modernisation 9
IV. COLLABORATION ET OUTILS MODERNES 10
4.1 La règle d'or : "Interdiction de toucher au Master" 10
4.2 La gestion des conflits (Simulation) 10
4.3 Modernisation des dépendances (Composer vs NPM) 11
4.4 L'automatisation avec Discord et les Bots 11
V. LE DEUXIÈME PROJET : MODULE AGENDA (MVC) 12
5.1 Un nouveau défi : L'architecture MVC 12
5.2 Intégration de l'éditeur Trumbowyg 12
5.3 La mission : Gestion visuelle des événements 13
5.4 Développement et Logique 14
5.5 L'étape critique : L'optimisation (Code Review) 15
ANNEXES 17
Annexe 1 : Exemple de code modernisé (Typage Strict) 17
Annexe 2 : Glossaire Technique 17

INTRODUCTION

Je suis actuellement étudiant en deuxième année de **BTS SIO**, option **SLAM** (Solutions Logicielles et Applications Métiers) au Lycée Louis Armand. Dans le cadre de ma formation, j'ai effectué un stage de huit semaines, du **15 décembre 2025 au 06 février 2026**.

J'ai choisi de réaliser ce stage au sein de l'association **Les Jardins Numériques**, située à Paris. Pourquoi ce choix ? Je cherchais une structure qui pouvait me proposer de vrais défis techniques. Lors de mes échanges, j'ai compris que j'allais travailler directement avec des experts et toucher à des technologies concrètes. Je ne voulais pas faire un stage "photocopieuse", je voulais coder pour de vrai.

L'objectif de ce stage était double : valider mes compétences acquises en cours, mais surtout découvrir la réalité du métier de développeur backend.

Ma mission principale consistait à **« consolider et moderniser la lib helpers PHP interne »**. Concrètement, l'association utilisait du code ancien (Legacy) qui fonctionnait, mais qui était difficile à maintenir, en raison du manque de structure du code. Mon rôle a été d'analyser ce code, de le nettoyer et de le réorganiser pour qu'il soit plus moderne et plus fiable.

Cette expérience a été un vrai changement pour moi. Au Lycée, j'avais l'habitude de travailler avec l'OS Windows avec des outils d'apprentissage. Ici, j'ai découvert un environnement professionnel exigeant. J'ai dû apprendre à utiliser le terminal **Git Bash**, à me connecter à des serveurs Linux via SSH et à gérer la sécurité des fichiers de programmation. Au début, c'était intimidant, mais j'ai rapidement progressé.

Ce rapport retrace mon parcours durant ces huit semaines. Il s'articule autour de quatre parties:

1. Je présenterai d'abord l'association **Les Jardins Numériques** et mon environnement de travail hybride.
2. Ensuite, j'expliquerai la mise en place technique de mon poste (passage de Windows à Linux).
3. La troisième partie détaillera ma mission de développement PHP (Refactoring).
4. Enfin, je parlerai de la collaboration en équipe et des outils comme Git.

Ce stage m'a permis de progresser techniquement et de confirmer mon envie de travailler dans le développement web.

I. PRÉSENTATION DE L’ASSOCIATION

1.1 Les Jardins Numériques : Une mission sociale et éducative

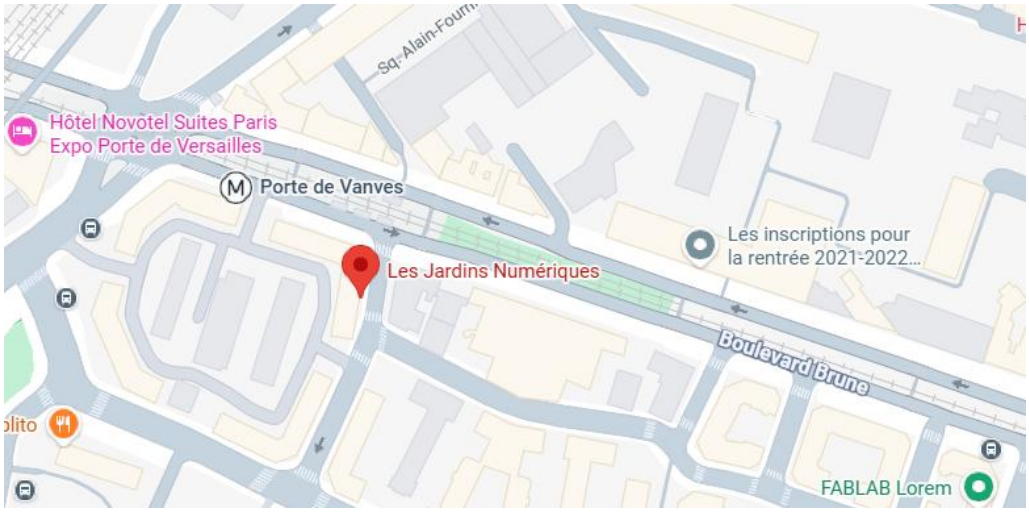
J'ai effectué mon stage au sein de l'association **Les Jardins Numériques**, située au 2 rue Wilfrid Laurier, dans le 17ème arrondissement de Paris.

C'est une structure (Loi 1901) qui a une philosophie très originale. Elle s'inspire du concept des "jardins partagés" pour créer des **Espaces Numériques Partagés (ENP)**. Son objectif principal est de réduire la fracture numérique et de rendre la technologie accessible à tous, sans distinction d'âge ou de milieu social.

L'association propose concrètement:

- Des ateliers d’initiation pour les débutants.
- Des formations adaptées (jeunes et adultes).
- Des projets collaboratifs avec d'autres acteurs locaux.

Cette organisation se fonde donc sur un environnement humain, basé sur l'entraide et l'échange.



1.2 L'équipe et l'organisation technique

Bien que la mission de l'association soit sociale, l'infrastructure technique qui la soutient est gérée avec une grande rigueur professionnelle.

L'équipe est restreinte et agile, ce qui m'a permis de travailler en direct avec les responsables :

- **M. Joseph HAN** (Président), qui pilote la vision globale et les partenariats.
- **M. Julien NAPOLI** (Consultant Technique et mon tuteur), qui assure la stabilité des outils numériques.

Mon rôle dans cette structure : L'association développe ses propres outils internes pour gérer ses activités. C'est sur cette partie que je suis intervenu. L'ambiance conviviale de l'équipe ("comme une pépinière d'associations") était associée à des exigences techniques pour le développement backend (au niveau du traitement des données) élevées.

J'ai travaillé sur le serveur de développement **`jnum14`** dans un environnement hybride (bureau et télétravail), en utilisant des outils de communication comme Discord pour rester synchronisé avec l'équipe.

II. MISE EN PLACE DE L'ENVIRONNEMENT TECHNIQUE

2.1 Un nouveau contexte : Du local au distant

Au lycée, j'avais l'habitude de travailler en "local" sur mon propre ordinateur avec XAMPP. Ici, j'ai découvert un environnement professionnel bien plus strict. L'association gère sa propre infrastructure. J'ai dû travailler sur un serveur de développement distant nommé **`jnum14`**.

Ce changement a été mon premier défi technique. Je ne pouvais plus simplement ouvrir des dossiers comme sous Windows. Tout devait passer par le réseau, de manière sécurisée.

2.2 Le défi de la connexion SSH

Pour accéder à ce serveur, l'authentification ne se faisait pas par un simple mot de passe, mais via une paire de clés de sécurité **SSH.**

Travaillant sur Windows, j'ai rencontré une difficulté majeure lors de la configuration. Le système accordait par défaut trop de permissions à ma clé privée, ce qui provoquait une erreur bloquante : *"**WARNING : UNPROTECTED PRIVATE KEY FILE**"*. J'ai dû apprendre à modifier les ACL (Access Control Lists) de Windows pour sécuriser ce fichier et réussir ma première connexion.

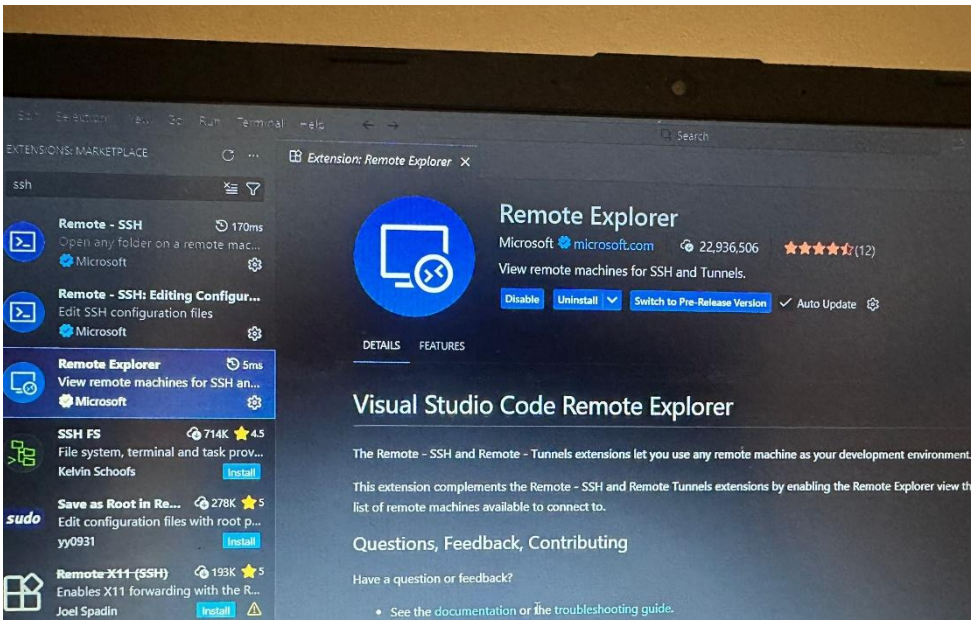
```
login as: shukdeb
Authenticating with public key "eddsa-key-20251204"
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-29-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Your Ubuntu release is not supported anymore.
For upgrade information, please visit:
http://www.ubuntu.com/releaseendoflife

New release '25.04' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Thu Dec  4 22:05:23 2025 from 78.197.126.234
shukdeb@jnagdpreprod:~$
```

2.3 L'environnement de développement : VS Code et Remote SSH

Une fois connecté, coder uniquement avec le terminal (via `nano`) s'est avéré complexe et peu ergonomique. Pour améliorer ma productivité, j'ai installé et configuré **Visual Studio Code** avec l'extension **"Remote - SSH"**.

C'était une véritable révélation. Cet outil m'a permis de :

1. Connecter mon éditeur directement au serveur `jnum14`.
2. Naviguer dans l'arborescence distante (`/var/www/jnhelpers`) comme si les fichiers étaient sur mon PC.
3. Modifier le code avec le confort d'un éditeur moderne.



2.4 La réalité du Terminal : Droits et Sudo

Malgré l'utilisation de VS Code, j'ai vite compris que la maîtrise du terminal restait indispensable pour l'administration du serveur. Lorsque j'ai voulu créer les dossiers du projet avec la commande `mkdir`, j'ai été bloqué par une erreur : **"Permission denied"**.

J'ai appris que sur un serveur Linux sécurisé, on ne peut pas tout faire n'importe comment. J'ai dû apprendre à utiliser la commande **`sudo`** pour obtenir les droits d'administrateur temporaires et débloquer la situation.

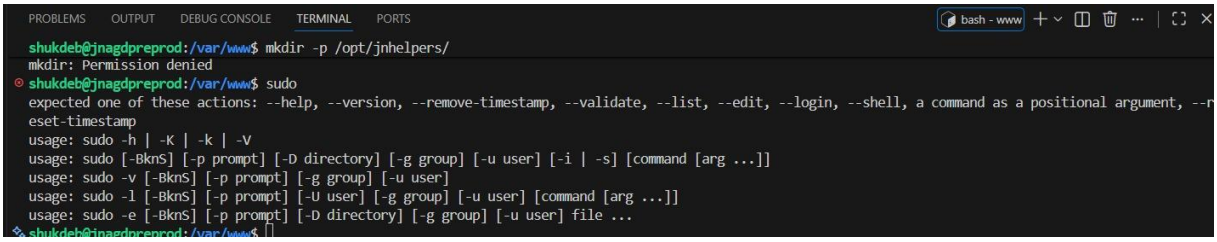


Figure 3 : Erreur de permission rencontrée et utilisation de la commande `sudo` pour la résoudre. Grâce à ces étapes, j'ai pu mettre en place un environnement de travail fonctionnel et professionnel.

III. DEVELOPPEMENT ET MODERNISATION

3.1 L'analyse du problème : L'héritage technique ("Legacy Code en PHP")

Une fois mon environnement prêt, mon tuteur m'a présenté le cœur du projet.

L'association utilisait depuis longtemps un fichier central nommé **`fonctions_html.php`**. À l'origine, ce fichier devait servir uniquement à l'affichage. Mais avec le temps, il s'est transformé en ce que mon tuteur a appelé un **"fourre-tout"**.

Pourquoi c'était un problème ? Ce fichier contenait des centaines de fonctions mélangées sans aucune logique :

- Du traitement de texte.
- Des calculs de dates.
- De la génération de code HTML.

C'était un code "monolithique". Si on voulait modifier une petite fonction, on risquait de casser une autre partie du site. La maintenance était devenue complexe et risquée.

3.2 Ma mission : "Diviser pour mieux régner"

Mon objectif n'était pas de créer de nouvelles fonctionnalités, mais de nettoyer l'existant. C'est ce qu'on appelle le **Refactoring**.

J'ai dû appliquer un principe clé du développement : la **Séparation des Résponsabilités** (Separation of Concerns). Concrètement, ma tâche consistait à :

1. **Analyser** les fonctions dans le dossier `_sources`.
2. **Identifier** leur rôle (ex : manipuler du texte, gérer le temps).
3. **Les déplacer** dans des fichiers dédiés et logiques.

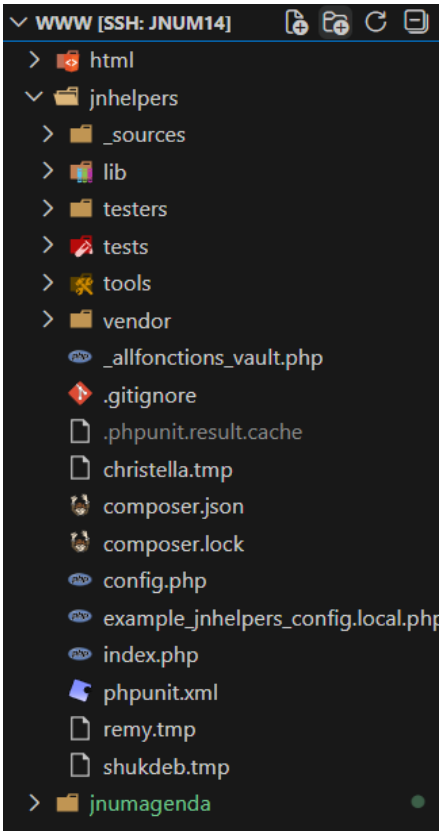


Figure : Nouvelle architecture du projet dans VS Code. On voit bien la séparation entre le dossier _sources (l'ancien code) et le dossier lib (le nouveau code organisé).

3.3 La réalisation technique : Typage et Modernisation

J'ai commencé par m'attaquer aux fonctions qui gèrent les chaînes de caractères (Strings). J'ai créé le fichier **`/lib/functions/functions_strings.php`**.

Mais je ne me suis pas contenté de faire du copier-coller. Le "vieux" code PHP n'était pas typé (il acceptait n'importe quelle donnée), ce qui est source de bugs. J'ai profité de cette migration pour moderniser la syntaxe en ajoutant le **Typage Strict** (Strict Typing).

Exemple de modernisation que j'ai réalisé :

- **Avant (Legacy) :** `function nettoyer($texte) { ... }` *(On ne sait pas si `$texte` est un mot ou un nombre.)*
- **Après (Mon code) :** `function nettoyer(string $texte): string { ... }` *(Je force l'entrée et la sortie en format texte. Le code devient plus robuste et sécurisé.)*

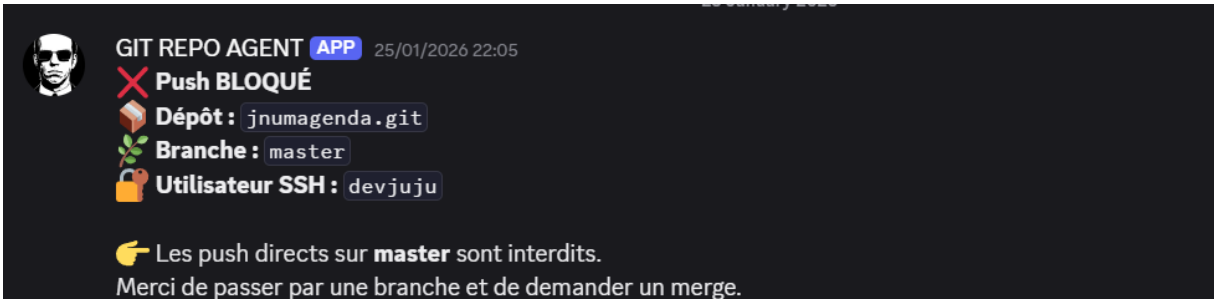
Ce travail a permis de rendre l'application plus stable et plus facile à comprendre pour les futurs développeurs.

IV. COLLABORATION ET OUTILS MODERNES

4.1 La règle d'or : "Interdiction de toucher au Master"

Dès le début, une règle de sécurité stricte m'a été imposée : **je n'avais pas la permission d'écrire directement sur la branche principale (`master`)**.

Si je tentais de contourner cette règle (même par erreur), le système de protection (Hook Git) rejetait immédiatement mon code avec un avertissement clair de notre bot de surveillance **GIT REPO AGENT** :



Push BLOQUÉ *Dépôt : jnumagenda.git | Branche : master "Les push directs sur master sont interdits. Merci de passer par une branche et de demander un merge."*

Cette restriction m'a forcé à adopter un workflow professionnel rigoureux :

```
shukdeb@jnagdpreprod:/var/www/jnhelpers$ git branch
  duplicat
  master
  test/datetime
  test/htmldisplay
  test/numbers
  test/strings
  test/temperature
* test/ui
shukdeb@jnagdpreprod:/var/www/jnhelpers$
```

1. Pour chaque tâche, je devais créer une nouvelle branche isolée (`git switch -b ma-tache`).
2. Je travaillais et testais mon code sur cette branche.
3. Une fois terminé, je "pushais" (« pousser le commit vers un dépôt Git Hub ») ma branche et mon tuteur s'occupait de la fusion vers le master.

4.2 La gestion des conflits (Simulation)

Le 24 décembre, pour tester ma compréhension de ce système, mon tuteur a organisé une simulation. Il a modifié le fichier `functions_warnings.php` sur le serveur pendant que je travaillais dessus.

Lors de ma tentative de mise à jour, j'ai rencontré un **"Merge Conflict"**. J'ai dû apprendre à :

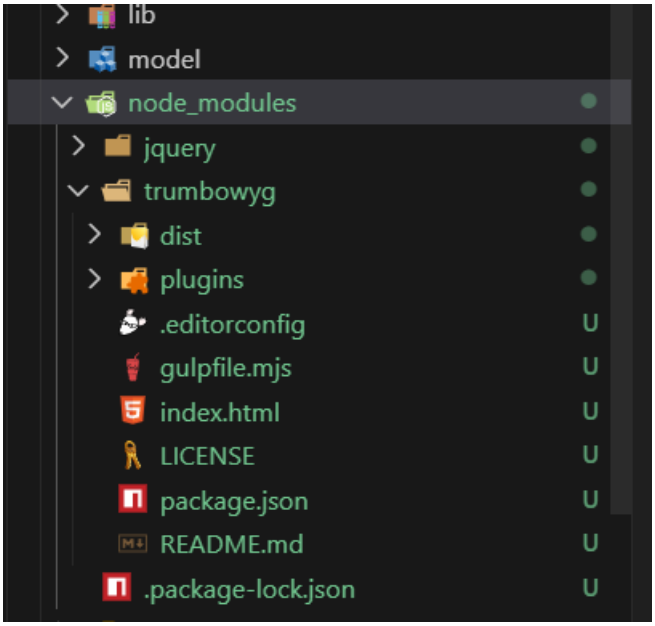
- Lire les marqueurs de conflit (`<<<<<<< HEAD`).
- Discuter avec l'équipe pour choisir la bonne version.
- Nettoyer le code avant de valider.

4.3 Modernisation des dépendances (Composer vs NPM)

Un autre aspect crucial de la modernisation du projet a été la gestion des librairies tierces. Dans les anciens projets PHP, on avait tendance à copier-coller les fichiers des librairies directement dans le projet, ce qui rendait les mises à jour difficiles.

J'ai appris à distinguer et utiliser deux gestionnaires de paquets standards :

1. **Composer (pour le Backend PHP) :** Nous l'avons utilisé pour gérer les dépendances côté serveur, comme *PHPUnit* pour les tests ou *PHP-CS-Fixer* pour le style de code. Grâce au fichier `composer.json`, il suffisait d'une commande (`composer install`) pour installer toutes les librairies nécessaires dans le dossier `/vendor`.



2. **NPM (pour le Frontend JS/CSS) :** Pour la partie interface, nous avons utilisé NPM (Node Package Manager). Cela nous a permis de gérer proprement les librairies JavaScript et CSS. Au lieu d'avoir des fichiers scripts éparpillés, tout était déclaré dans `package.json` et centralisé dans le dossier `/node_modules`.

Cette séparation claire m'a permis de mieux comprendre l'architecture d'un projet web moderne, où le Backend et le Frontend ont chacun leur propre cycle de vie.

4.4 L'automatisation avec Discord et les Bots

La communication au sein de l'équipe était fluidifiée par des outils d'automatisation connectés à notre serveur Discord. Nous utilisions deux bots spécifiques pour surveiller l'activité du projet:

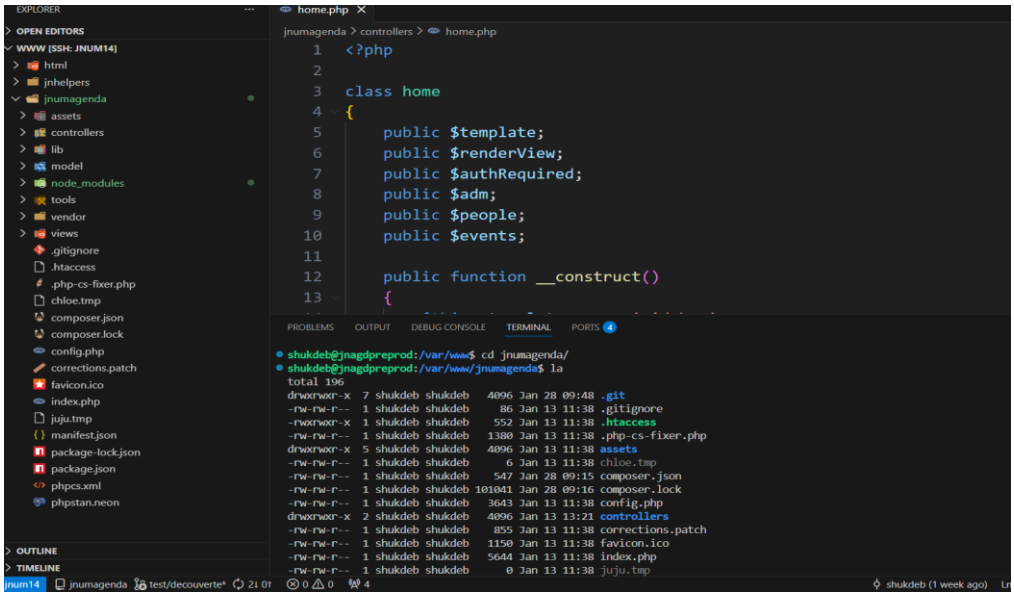
- **GIT REPO AGENT :** Ce bot nous notifiait en temps réel à chaque fois qu'un membre de l'équipe effectuait un "Commit" ou un "Push". Cela permettait de suivre l'avancement des autres sans avoir à leur demander.
- **Bot-icelli :** Un autre assistant numérique (utilisé sur le projet *jnhelpers*) qui facilitait la gestion des tâches et des notifications sur le serveur.

V. LE DEUXIÈME PROJET : MODULE AGENDA (MVC)

5.1 Un nouveau défi : L'architecture MVC

En janvier, une fois la librairie de fonctions modernisée, mon tuteur m'a confié une nouvelle mission plus complexe. Il s'agissait de travailler sur le **"Site Agenda"**, une application interne permettant de gérer les événements de l'association.

Ce projet était techniquement différent du premier. Ici, nous n'étions plus sur du code

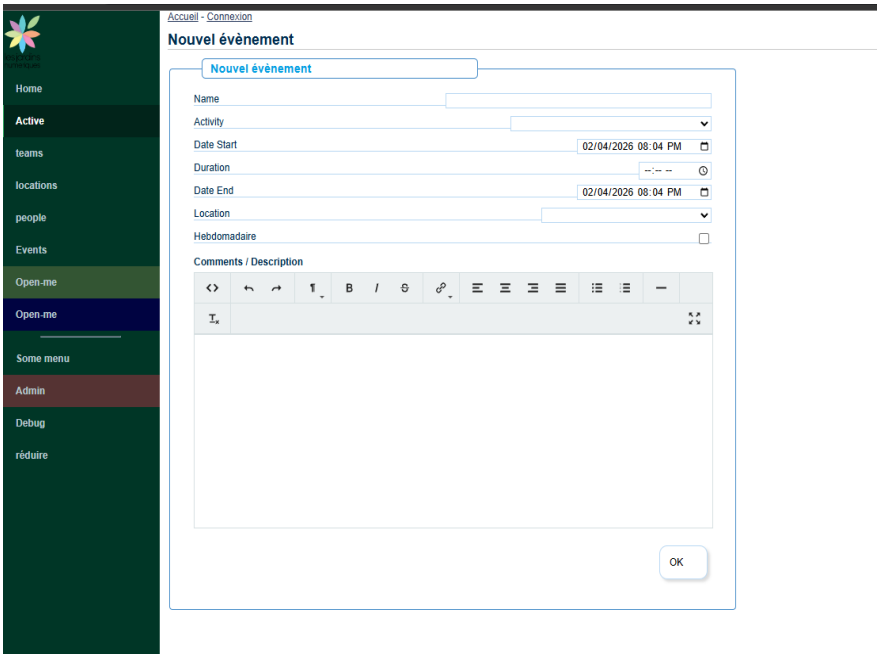


procédural simple, mais sur une architecture **MVC (Modèle - Vue - Contrôleur)** avec de la programmation orientée objet (PHP POO).

C'était un défi pour moi car je devais comprendre comment les fichiers interagissaient entre eux (Le Contrôleur appelle le Modèle, qui envoie les données à la Vue).

5.2 Intégration de l'éditeur Trumbowyg

Dans le cadre de l'amélioration de l'interface utilisateur (notamment pour l'administration des événements), il a fallu remplacer les simples zones de texte (`<textarea>`) par un éditeur de texte riche (WYSIWYG).



Le choix s'est porté sur **Trumbowyg**, un éditeur léger et performant. Mon travail a consisté à :

1. Installer la librairie via NPM pour garantir qu'elle soit à jour.
2. Intégrer les fichiers JS et CSS dans le *header* de notre application.
3. Initialiser l'éditeur sur les champs de description avec du code JavaScript/jQuery.

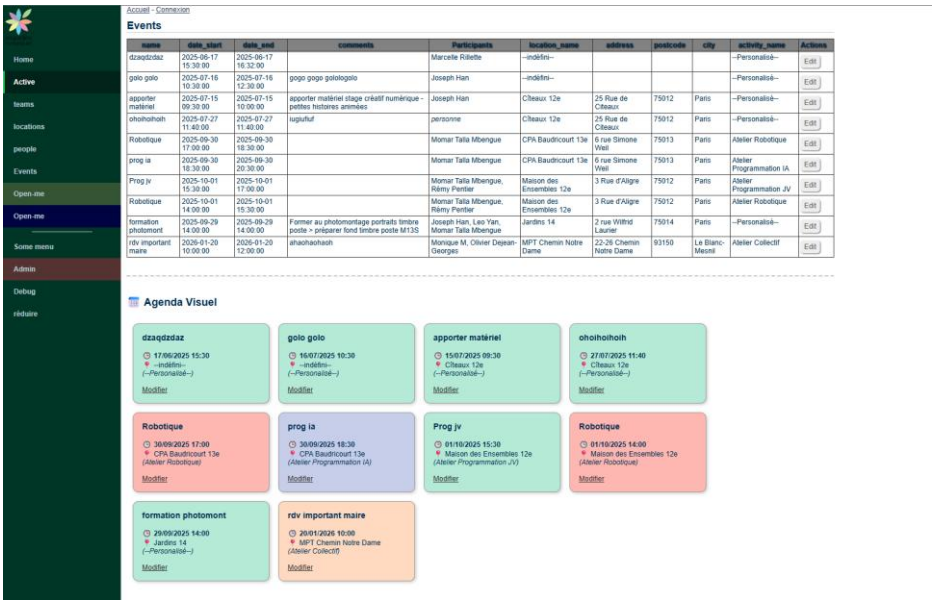
Objectif : Permettre aux utilisateurs de mettre en forme leur texte (gras, italique, listes) sans avoir à écrire du code HTML, améliorant ainsi considérablement l'expérience utilisateur (UX).

5.3 La mission : Gestion visuelle des événements

Ma tâche spécifique était d'améliorer l'affichage des événements sur la page d'accueil (`home_default.php`) pour l'utilisateur connecté. Le besoin était précis : il fallait différencier visuellement les événements en fonction de leur date.

J'ai dû développer une logique en PHP pour comparer la date de l'événement avec la date d'aujourd'hui (`datetime`), et appliquer une classe CSS spécifique.

Le résultat attendu :



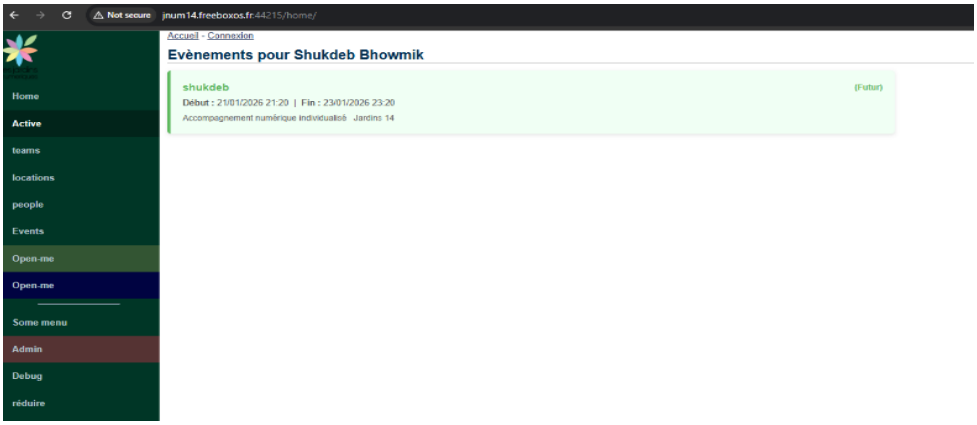
- **Rouge :** Les événements passés.
- **Bleu :** Les événements du jour (Aujourd'hui).
- **Vert :** Les événements futurs.

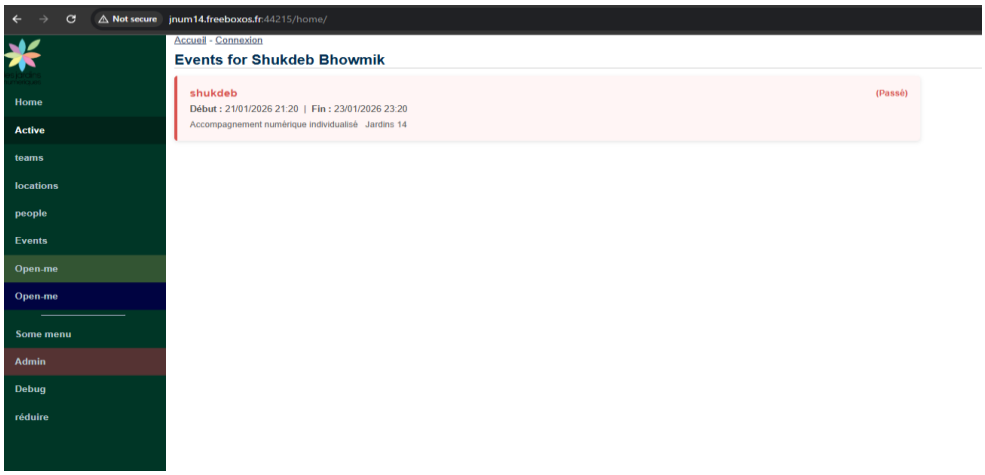
5.4 Développement et Logique

Pour réaliser cela, j'ai dû :

1. Récupérer la liste des événements via le Contrôleur.
2. Créer une boucle `foreach` pour parcourir chaque événement.
3. Ajouter une condition `if / else` pour vérifier la date.

Extrait de la logique que j'ai mise en place : "Si la date de l'événement est < maintenant -> Alors couleur = rouge" "Si la date de l'événement est > maintenant -> Alors couleur = vert"





Cette mission m'a permis de manipuler des objets PHP et de comprendre concrètement l'intérêt du MVC pour séparer la logique (PHP) du design (HTML/CSS).

5.5 L'étape critique : L'optimisation (Code Review)

Une fois l'affichage fonctionnel, j'ai soumis mon code à mon tuteur pour validation. C'est là que j'ai appris une leçon importante sur la performance.

J'avais écrit une méthode pour afficher le titre de la page en faisant une requête SQL inutile :

Mon code initial: `$this->people = new model_people();`

```
$person = $this->people->getOnePeople($peopleId);
```

Mon tuteur m'a expliqué que c'était une erreur : les informations de l'utilisateur connecté étaient déjà disponibles en session via la classe `auth`. Il n'y avait pas besoin d'interroger la base de données à nouveau.

Code optimisé (Après correction):

```
$obj->pageTitle = 'Events for ' . auth::getConnectedMemberName();
```

Cette correction m'a permis d'alléger l'application et de comprendre l'importance de bien connaître les outils existants du framework avant de réécrire des fonctions.

VI. CONCLUSION ET BILAN PERSONNEL

6.1 Bilan Technique : Une transformation concrète

Ce stage de huit semaines a été une étape décisive dans mon parcours. Je suis passé d'un environnement scolaire (Windows, XAMPP) à un environnement professionnel exigeant (Linux, SSH, MVC).

J'ai acquis des compétences solides sur :

- **L'administration système :** Je n'ai plus peur du terminal. Je sais gérer des clés SSH et des permissions Linux.
- **Le développement PHP moderne :** J'ai appris le "Typage Strict" et l'architecture MVC.
- **La gestion de version (Git) :** Je ne vois plus Git comme une simple sauvegarde, mais comme un outil de collaboration sécurisé (Branches, Merge Requests).

6.2 Bilan Professionnel

Au-delà de la technique, mon tuteur M. Napoli m'a enseigné la rigueur. J'ai retenu trois leçons qui me suivront :

1. **La précision :** Ne jamais dire *"ça ne marche pas"*, mais analyser les logs pour expliquer pourquoi.
2. **La sécurité :** Ne jamais travailler sur la branche master.
3. **L'autonomie :** Ne pas rester bloqué face à une erreur inconnue, mais savoir décomposer le problème pour trouver une solution technique adaptée.

CONCLUSION GÉNÉRALE

L'expérience au sein de l'association **Les Jardins Numériques** m'a confirmé que j'ai fait le bon choix d'orientation. Le développement backend est exigeant, mais incroyablement gratifiant quand on trouve la solution.

Je me sens désormais prêt à poursuivre mes études et à relever de nouveaux défis techniques avec confiance.

ANNEXES

Annexe 1 : Exemple de code modernisé (Typage Strict)

Voici un extrait du fichier `functions_strings.php` après mon intervention. On y voit l'application du typage strict (int, string) qui n'existait pas dans l'ancienne version.

```
<?php

// Nettoyage basique des espaces

function cleanString(string $text): string

{

    if (empty($text)) {

s        return "";

    }

    return trim(preg_replace('/\s+/', ' ', $text));

}

// Vérifie si une chaîne contient un mot spécifique.

function containsWord(string $haystack, string $needle): bool

{

    return strpos($haystack, $needle) !== false;

}
```

Annexe 2 : Glossaire Technique

Liste des termes techniques et acronymes utilisés dans ce rapport.

- **CLI (Command Line Interface) :** Interface permettant d'interagir avec le système via des commandes textuelles (ex: Terminal Linux).

- **SSH (Secure Shell) :** Protocole de communication sécurisé utilisé pour se connecter au serveur `jnum14` à distance.
- **MVC (Modèle-Vue-Contrôleur) :** Architecture logicielle séparant les données (Modèle), l'interface (Vue) et la logique (Contrôleur).
- **Refactoring :** Processus consistant à modifier la structure interne d'un code sans changer son comportement externe, pour le rendre plus propre et maintenable.
- **Git Flow :** Méthodologie de gestion des branches Git (ex: ne jamais commit sur master directement).
- **Legacy Code :** Code source hérité d'anciennes versions du logiciel, souvent obsolète ou difficile à maintenir.